DTS 201- Some Data Modeling Algorithms

Idowu, G. A.

University of Lagos, Akoka

February 11, 2026

Algorithm Selection Framework

- **Problem Type** → Supervised/Unsupervised/Semi-supervised/Reinforcement
- **Target Variable** → Continuous/Discrete/Ordinal/No target
- **Feature Characteristics** → Numerical/Categorical/Text/Image/Sequence
- **Data Volume** → Small (<1k)/Medium (<100k)/Large (<100k)/Massive (>1M)
- **Interpretability Need** → Must explain vs. Black-box acceptable
- **Resource Constraints** → Training time/Prediction latency/Memory

1. Linear Regression

Structure:

$$Y = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n + \varepsilon$$

Strengths:

- ✓ Most interpretable
- ✓ Closed-form solution
- ✓ Low variance

Weaknesses:

- × Fails with non-linearity
- × Sensitive to outliers
- × Multicollinearity issues

Example: House price prediction from sq footage, bedrooms, lot size

2. Ridge & Lasso Regression

Regularized Regression:

- **Ridge (L2):** $\min \sum (Y - \hat{Y})^2 + \lambda \sum \beta_j^2$
- **Lasso (L1):** $\min \sum (Y - \hat{Y})^2 + \lambda \sum |\beta_j|$
- **Elastic Net:** Combines both

Strengths:

- ✓ Prevents overfitting
- ✓ Lasso: feature selection
- ✓ Ridge: handles multicollinearity

Weaknesses:

- × Requires λ tuning
- × Features must be scaled
- × Lasso selects arbitrarily among correlated

Example: Genomics - drug response from 20,000 gene expressions

3. Logistic Regression

Structure:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \ldots)}}$$

Strengths:

- ✓ Well-calibrated probabilities
- ✓ Fast training/prediction
- ✓ Interpretable coefficients

Weaknesses:

- × Linear decision boundary
- × No automatic interactions
- × Sensitive to correlated features

Example: Credit default prediction - P(Default) = 0.73

4. K-Nearest Neighbors (KNN)

Structure:

1. Store entire training dataset
2. Compute distance to all training points
3. Select K closest points
4. Majority vote (classification) or average (regression)

Strengths:

- ✓ No training phase
- ✓ Complex decision boundaries
- ✓ Simple to implement

Weaknesses:

- × Slow prediction O(n)
- × Curse of dimensionality
- × Sensitive to irrelevant features

Example: Movie recommender - find similar users

5. Support Vector Machine (SVM)

Kernel Trick:

- Linear: $K(x, y) = x \cdot y$
- Polynomial: $K(x, y) = (x \cdot y + r)^d$
- RBF: $K(x, y) = \exp(-\gamma \|x - y\|^2)$

Strengths:

✓ Effective in high-dimensions

✓ Memory efficient

✓ Versatile kernels

Weaknesses:

× Poor with overlapping classes

× No direct probabilities

× $O(n^2)$ to $O(n^3)$ training

Example: MNIST digit recognition - 784 features

6. Decision Tree (CART)

Split Criteria:

- Gini: $1 - \sum p_i^2$
- Entropy: $-\sum p_i \log_2(p_i)$
- MSE (regression)

Strengths:

✓ Highly interpretable

✓ No scaling needed

✓ Handles mixed data

Weaknesses:

× High variance

× Prone to overfitting

× Greedy algorithm

Example: Loan approval system - IF-THEN rules

7. Random Forest

Structure:

- Bootstrap aggregating (bagging)
- Random feature selection at splits
- Majority vote / average
- Out-of-bag (OOB) error estimation

Strengths:

✓ High accuracy

✓ Resistant to overfitting

✓ Feature importance

Weaknesses:

× Black box

× Large memory footprint

× Slower prediction

Example: Customer churn - top predictors identified

8. Gradient Boosting (XGBoost/LightGBM/CatBoost)

Sequential Learning:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Strengths:

- ✓ State-of-the-art for tabular
- ✓ Handles missing values
- ✓ Customizable loss

Weaknesses:

- × Many hyperparameters
- × Sensitive to outliers
- × Can overfit

Example: Airbnb price prediction - tuned parameters shown

9. Naive Bayes

Bayes Theorem:

$$P(C|F) = \frac{P(C) \times \prod P(F_i|C)}{P(F)}$$

Assumption: Features independent given class

Strengths:

- ✓ Extremely fast
- ✓ Works with high-dimensions
- ✓ Small data performance

Weaknesses:

- × Independence assumption
- × Zero probability problem
- × Not for regression

Example: Email spam classification - "viagra", "winner"

10. Neural Networks (Deep Learning)

Neuron: $z = \sum(w_i x_i) + b$, $a = \sigma(z)$

- **CNN:** Images
- **RNN/LSTM:** Sequences
- **Transformer:** NLP

Strengths:

✓ Universal approximators

✓ Feature hierarchies

✓ SOTA for unstructured

Weaknesses:

× Needs large data

× Black box

× Computationally expensive

Example: Pneumonia detection from chest X-rays

11. K-Means

Objective: $\min \sum \|x - \mu_c\|^2$

1. Initialize K centroids
2. Assign points to nearest centroid
3. Update centroids
4. Repeat until convergence

Strengths:

- ✓ Simple, scalable O(n)
- ✓ Guaranteed convergence

Weaknesses:

- × Must specify K
- × Spherical clusters only
- × Sensitive to outliers

Example: Customer segmentation (4 segments: Premium, Aspirational, Conservative, Budget)

12. Hierarchical Clustering

Types:

- Agglomerative (bottom-up): merge closest
- Divisive (top-down): recursively split

Linkage: Single, Complete, Average, Ward

Strengths:

✓ No need to specify K

✓ Dendrogram visualization

Weaknesses:

× $O(n^3)$ complexity

× Not scalable

Example: Phylogenetic tree - evolutionary relationships

13. DBSCAN

Density-based clustering:

- Core: $\geq$ MinPts within ε
- Border: $<$ MinPts but reachable
- Noise: Not reachable

Strengths:

✓ Arbitrary shapes

✓ Robust to outliers

✓ No need for K

Weaknesses:

× Sensitive to ε

× Varying density issues

× High-dimension fails

Example: Crime hotspot detection - dense clusters vs isolated incidents

14. Principal Component Analysis (PCA)

Steps:

1. Standardize data
2. Compute covariance matrix
3. Eigendecomposition
4. Project to top k components

Strengths:

- ✓ Removes multicollinearity
- ✓ Noise reduction
- ✓ Visualization

Weaknesses:

- × Linear only
- × Not interpretable
- × Sensitive to scaling

Example: MNIST: 784 $\rightarrow$ 2D, 95% variance with 154 components

15. t-SNE

Non-linear visualization:

- Convert distances to probabilities
- Student-t distribution in low-D
- Minimize KL divergence

Perplexity: 5-50 (balance local/global)

Strengths:

✓ Reveals clusters

✓ Preserves local structure

✓ Excellent visualization

Weaknesses:

× Non-deterministic

× Not for feature extraction

× $O(n^2)$ complexity

Example: Word embeddings - semantic relationships visualized

16. Apriori Algorithm

Metrics:

- Support: $P(\text{Itemset})$
- Confidence: $P(Y|X)$
- Lift: $\frac{P(X \cup Y)}{P(X)P(Y)}$

Apriori principle: All subsets of frequent itemset are frequent

Strengths:

✓ Exhaustive search

✓ Interpretable rules

Weaknesses:

× Very slow

× Multiple database scans

Example: Diapers → Beer, Support=2%, Confidence=60%, Lift=2.5

17. FP-Growth

Structure:

1. FP-tree construction (2 scans)
2. Recursive mining with conditional pattern bases

Strengths:

- ✓ Much faster than Apriori
- ✓ Only 2 database scans
- ✓ Compact structure

Weaknesses:

- × Memory intensive
- × Complex implementation

Example: Web clickstream - 50M sessions, 100K pages compressed to 2M nodes

18. ARIMA

Components:

- AR(p): $Y_t = c + \phi_1 Y_{t-1} + \ldots + \varepsilon_t$
- I(d): d differences for stationarity
- MA(q): $Y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \ldots$

Seasonal ARIMA: (p,d,q)(P,D,Q)$_m$

Strengths:

- ✓ Statistical foundation
- ✓ Interpretable
- ✓ Prediction intervals

Weaknesses:

- × Requires stationarity
- × Linear only
- × No exogenous variables

Example: Monthly airline passengers - ARIMA(0,1,1)(0,1,1)$_{12}$

19. LSTM (Long Short-Term Memory)

Gates:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Strengths:

- ✓ Long-term dependencies
- ✓ Solves vanishing gradient
- ✓ Variable-length sequences

Weaknesses:

- × Requires large data
- × Computationally expensive
- × Black box

Example: Stock prediction with news sentiment

20. Isolation Forest

Key insight: Anomalies are few and different $\rightarrow$ easier to isolate

$$s(x, n) = 2^{-E(h(x))/c(n)}$$

- $s \approx 1$: Definite anomaly
- $s < 0.5$: Normal point

Strengths:

✓ Linear time O(n)

✓ No distance computation

✓ Handles high-dimensions

Weaknesses:

× Assumes global anomalies

× Threshold subjective

× Drops with irrelevant features

Example: Credit card fraud - 0.1% fraud rate, real-time detection

Algorithm Selection by Target Type

Target Type	Primary	Avoid
Continuous	Linear/Ridge, RF, XGBoost	Naive Bayes
Binary	Logistic, XGBoost, RF	Linear Regression
Multiclass	RF, XGBoost, Neural Nets	SVM (expensive)
Clustering	K-Means, DBSCAN	Classification algos
Images	CNN, Transfer Learning	RNN, ARIMA
Time Series	ARIMA, LSTM, Prophet	Standard regression

Algorithm Trade-offs

Algorithm	Accuracy	Speed	Interpret	Data Volume
Linear Regression	Low	5/5	5/5	Small
Random Forest	Very High	2/5	2/5	Medium-Large
XGBoost	Very High	2/5	2/5	Medium-Large
Neural Networks	High	1/5	1/5	Large-Very Large
K-Means	N/A	4/5	4/5	Large
DBSCAN	N/A	2/5	3/5	Small-Medium

Example 1: Customer Churn Prediction

Dataset: 100K customers, 50 features, 15% churn rate

Algorithm	**Accuracy**	**F1**	**AUC**	**Time**
Logistic	0.81	0.60	0.79	2*sec*
Decision Tree	0.77	0.54	0.68	5*sec*
Random Forest	0.86	0.69	0.88	45*sec*
XGBoost	**0.88**	**0.72**	**0.91**	60*sec*

Winner: XGBoost with class weights

Key insight: Number of customer service calls is strongest predictor

Example 2: Fraud Detection

Dataset: 1M transactions, 30 features, 0.1% fraud rate
Requirement: <100ms latency

Algorithm	Capture Rate	FPR	Latency	Size
Logistic	67%	3.2%	$< 1ms$	$2KB$
Random Forest	84%	1.8%	$15ms$	$850MB$
XGBoost	**89%**	**1.5%**	$8ms$	$120MB$
Isolation Forest	72%	4.1%	$5ms$	$45MB$

Winner: XGBoost (accuracy-latency balance)

Example 3: Image Classification

Dataset: 10K chest X-rays, pneumonia detection

Architecture	Params	Accuracy	AUC	Time
CNN from scratch	$2.1M$	0.82	0.88	$3h$
ResNet50 (pretrained)	$25.6M$	0.89	0.94	$1h$
ResNet50 (fine-tuned)	$25.6M$	**0.92**	**0.96**	$2h$
DenseNet121	$8.0M$	0.91	0.95	$2.5h$

Winner: Fine-tuned ResNet50

Example 4: Market Basket Analysis

Dataset: 500K transactions, 10K products

Metric	Apriori	FP-Growth
Execution time	45 min	**2.5 min**
Memory usage	8 GB	**3.2 GB**
Database scans	11	**2**

Winner: FP-Growth (18x faster)
Top rule: {Peanut butter, Bread} $\rightarrow$ {Jelly}
Support: 0.8%, Confidence: 68%, Lift: 3.2

Example 5: Customer Segmentation

Dataset: 50K customers, RFM features

Algorithm	**Silhouette**	**Interpretability**	**Outliers**
K-Means	0.52	4/5	*Poor*
Hierarchical	0.51	5/5	*Poor*
DBSCAN	**0.61**	3/5	*Excellent*

Winner: DBSCAN

Segments: Loyal premium, Recent bargain hunters, At-risk high value, Dormant

Algorithm Selection Flowchart

START: Classification Task

```
|
|– Is data tabular?
   |– YES → Interpretability critical?
      |– YES → n < 10K? → Decision Tree/Logistic
      |– NO → n > 100K? → XGBoost/Neural Nets
   |– NO → (Images/Text/Time Series)
      |– Images → CNN/Transfer Learning
      |– Text → Transformers/LSTM/Naive Bayes
      |– Time Series → LSTM/ARIMA
```

Do's and Don'ts

Scenario	Do	Don't
Small dataset (¡1k)	Simple models + regularization	Neural nets, 1000-tree ensembles
Imbalanced	Class weights, SMOTE	Accuracy metric
High-dimensional	Lasso, RF, PCA first	KNN, DBSCAN
Missing values	XGBoost, RF, impute	SVM, NN without imputation
Categorical high cardinality	Target encoding, CatBoost	One-hot encoding
Time series	Temporal CV	Random train/test split

Recommended Workflow

1. **Start simple:** Linear/Logistic regression (interpretable baseline)
2. **Try Random Forest:** Robust, good performance, minimal tuning
3. **Optimize with XGBoost/LightGBM:** State-of-the-art for structured data
4. **Deep Learning:** Only for unstructured data or massive datasets
5. **Always cross-validate** and compare multiple algorithms

No single best algorithm - depends on data, constraints, and requirements!

Key Takeaways

- **Understand your data** before choosing algorithms
- **Interpretability vs. Performance** trade-off is fundamental
- **Ensemble methods** (RF, XGBoost) dominate structured data
- **Deep Learning** excels at unstructured data (images, text, audio)
- **Scale features** for distance-based algorithms
- **Handle missing values** appropriately
- **Use cross-validation** for honest performance estimates
- **Start simple, then iterate**